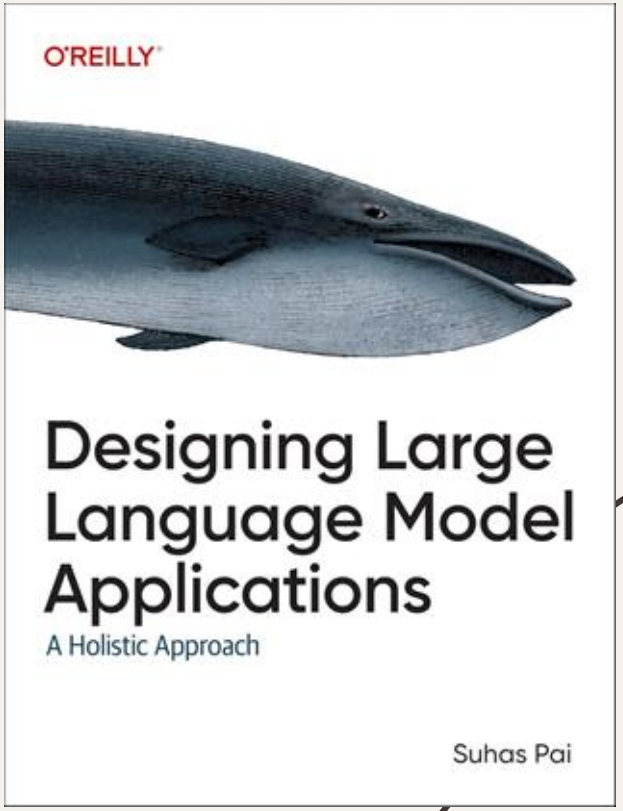
Designing Agent Architectures

- Suhas Pai, CTO, Hudson Labs

About Me

- Co-founder, CTO, & ML Research @ Hudson Labs (https://hudson-labs.com)
- Book ‘Designing LLM Applications’ published by O’Reilly Media
- Led/contributed to various open-source LLMs (BLOOM, Aurora-M etc.)
- Chair, TMLS (Toronto Machine Learning Society) conference, 2021-present
- Co-founder of independent ML research org Fruitless Labs



Course Modules

- Module 1: Defining Agents and Autonomy
- Module 2: Components of an Agent
- Module 3: Memory and Retrieval
- Module 4: Design Patterns

Module 1

Defining Agents & Autonomy

Defining Agents - Colloquial definition

Software systems enacting a sequence of actions to automate a task

Defining Agents - Colloquial definition

Defining Agents - our definition

‘**LLM-driven** software systems that are able to **interact with their environment** and take **autonomous actions** to complete a task.’

Key aspects of our definition

- Interaction with environment - the system is able to connect and communicate with external data and software to achieve its goals.
- Autonomous actions - the decision to perform an action is not made by a human

An agent’s scope is determined by the level of autonomy and access it possesses.

Levels of Autonomy

- Traditional Automation
- LLM-enhanced application
- Code-orchestrated LLM workflow
- Autonomous agent

Passive Approach - LLM-enhanced application

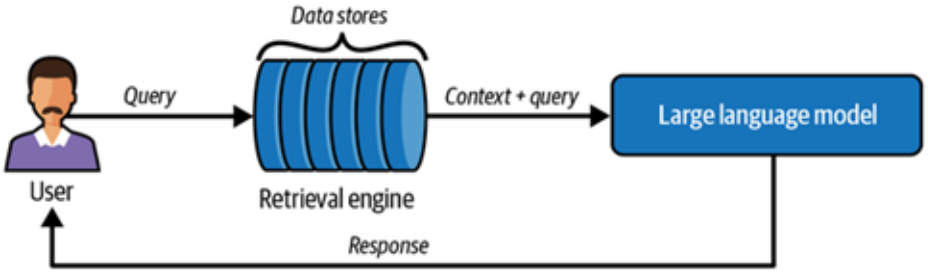


Figure 10-1. An LLM passively interacting with a data store

Passive Approach

- Highly specific and deterministic workflows
- LLM need not interact with its environment
- LLM call is the last and only step in the workflow

Examples:

Question Answering system over a knowledge base

Summarize or format data fetched from a database

Discussion

What are some use cases that you have worked on that can be implemented using the passive approach?

Explicit Approach - Code orchestrated LLM workflow

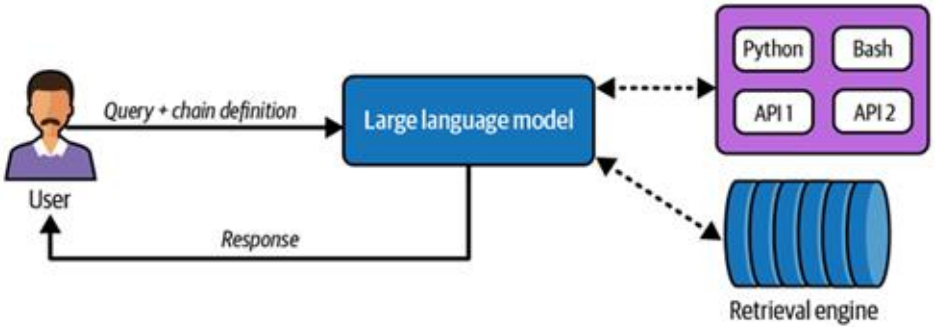


Figure 10-2. The explicit interaction approach in action

Explicit Approach

- Sequence of actions that need to be taken for a particular task is known in advance
- Type of tasks supported are limited and known in advance

Examples:

1. When stock price goes up by over 5 percent, query the web to find out the reasons, perform a deeper analysis on each of the reasons and provide a summarized output.
2. When provided with a user query, generate a SQL query to fetch data, then use the data to perform data analysis and visualizations.

Discussion

What are some use cases that you have worked on that can be implemented using the explicit approach?

Autonomous Approach

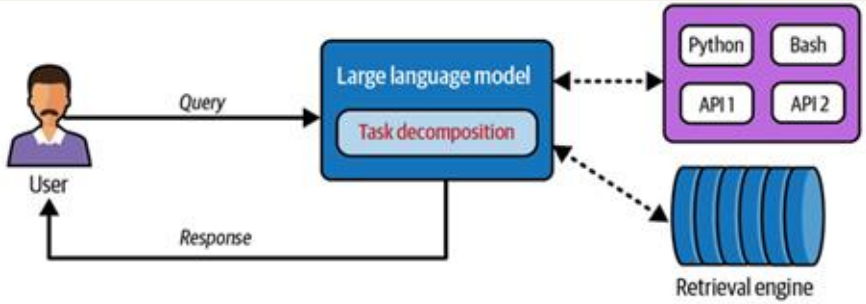


Figure 10-4. A typical autonomous LLM-driven agent workflow

Autonomous Approach

- The LLM makes autonomous decisions about whether and when to interact with its environment (data/software)
- Can support very general queries

Examples:

1. Perform ‘deep research’ on the causes of the decline of the Mughal dynasty
2. Plan and book a travel itinerary for a family of 3 with a specified budget and destination list

Discussion

What are some use cases that you have worked on that can be implemented using the autonomous approach?

Caution

Agents still struggle over long-horizon tasks involving many steps.

Caution

Be wary of providing LLMs unfettered access to commit actions - a security nightmare

Autonomous Approach - An Example

Example Query:

> ‘Who was the prime minister of Italy during the year when Italy had its highest yearly GDP growth in the last 15 years?’

Autonomous Approach - An example

1. Find current date and calculate the date range
2. Find database table with GDP numbers and possibly calculate growth rates
3. Find list of Italian prime ministers and their tenures
4. Match highest growth rate year with tenure list to find the prime minister(s)

Note: There are multiple ways to solve this problem

Question

Which of these is an agent according to our definition?

- Tool that sends an email whenever stock price of a company is up by 5% in a single day
- Tool that books an airline ticket when requested to do so in natural language
- Tool that summarizes daily news and sends a Whatsapp message everyday with the headlines
- Tool that anticipates your day-to-day needs and sends you reminders on your phone (don’t forget your umbrella etc)

Components of a typical agentic system

- Models
- Data Stores
- Tools
- Agent Loop prompt
- Orchestration software (harness)
- Guardrails and Verifiers

Components of a typical agentic system

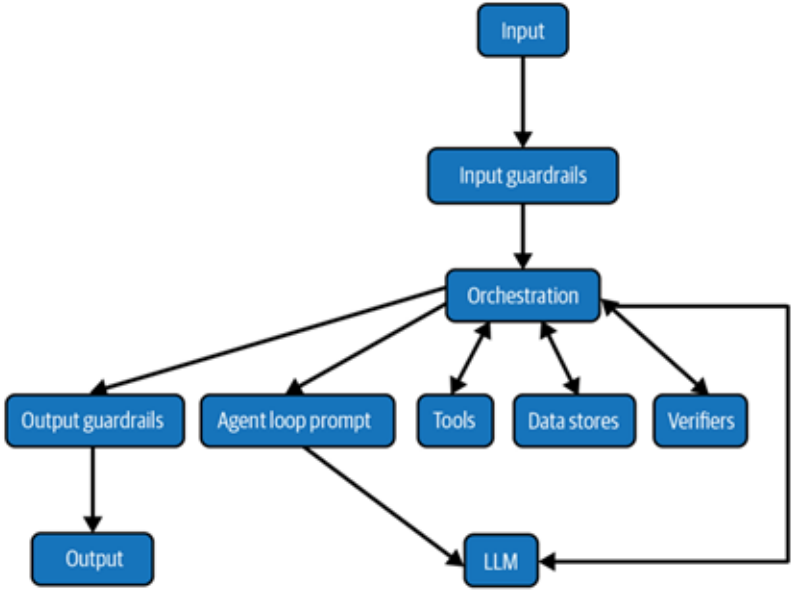


Figure 10-5. A production-grade agentic system

Discussion

What agentic frameworks have you heard of or are familiar with?

Popular agent frameworks

- LangGraph
- LlamaIndex
- Haystack
- CrewAI
- OpenAI Agents SDK
- Google Agent Development Kit
- Microsoft Agent Framework
- Claude Agent SDK
- Pydantic AI
- AG2

Exercise

Build a rudimentary agent using an existing agentic framework. The agent should have access to the Web and some free APIs.

Module 2

Components of an agent & their evaluation

Models

- Pick models that are post-trained for effective tool use
- Use multiple models if needed, differentiated by size and/or specialization
- Characteristics to watch out for: context window size, native tool support, computer use, structured outputs

Model Benchmarks

- **GAIA**: broad assistant capability
- **BrowseComp**: deep web research
- **WebArena**: interactive web/UI grounding
- **OSWorld**: real computer-use
- **SWE-bench Verified or Live**: codebase modification
- **Terminal-Bench**: terminal/devops/data tasks
- **AgentHarm or Agent-SafetyBench**: safety
- **HCAST / METR time horizon**: long-horizon autonomy

Multi-model example

- When the output needs to be in a highly specific format, it impacts the overall quality of the answer.
- Solution: fine-tune a small model whose only job is to take an intermediate output and convert it into the highly specific output format
- Tradeoff: can’t stream the intermediate solution, time-to-first-token is affected

Multi-model example

- When code needs to be generated, delegate it to a coding model
- When the model needs to reason, a large reasoning model can be employed

Discussion

What are your favorite models? Do you use open-source/open-weights models?

Multi-model setup

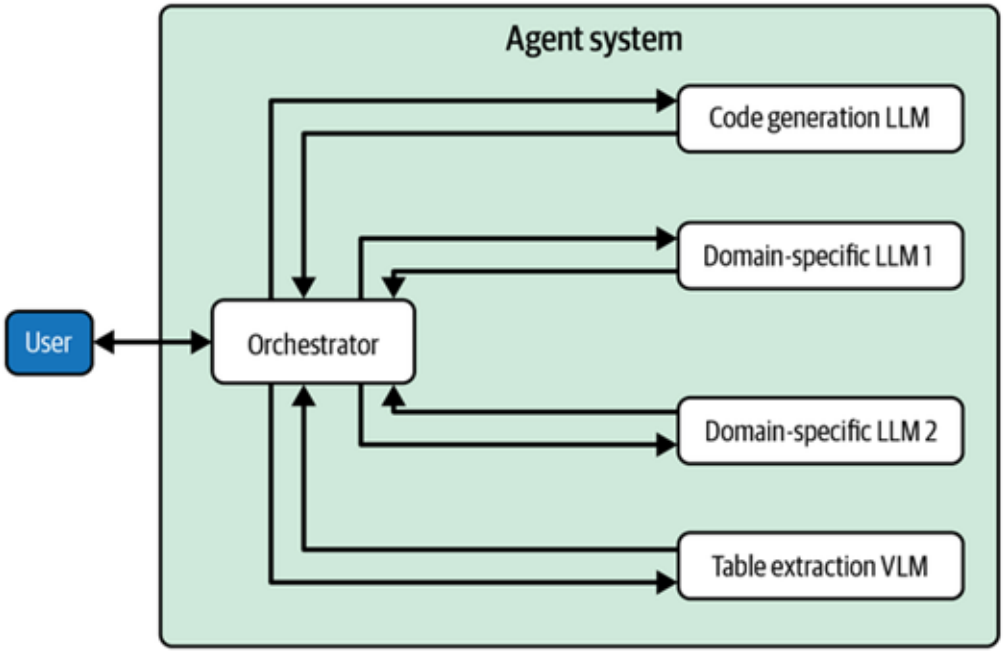


Figure 10-6. An agentic system with multiple LLMs

Models - Trade offs

- Cost, Latency vs Performance
- Ability to route to the correct model for a given workflow step

Tools

- Connect to the Web
- Connect to APIs
- Connect to databases
- Connect to code interpreter

Implementing tools

- Typed input and output
- Preconditions and postconditions
- Read-only versus write
- Reversible versus irreversible
- Idempotent versus non-idempotent
- Expected error types
- Required authorization scope
- Approval requirements
- Timeout and retry policy

MCP (Model Context Protocol)

- Equivalent of a USB-C port for AI applications
- Build once, integrate everywhere
- Exposes APIs, files, databases etc

MCP (Model Context Protocol)

- MCP Hosts - ChatGPT, Cursor etc
- MCP Server - Weather.com, Yahoo Finance, Github etc

MCP Primitives

- Tools (API calls, database queries etc)
- Resources (database schemas etc)
- Prompts

Discussion

Why is MCP useful? Why can’t we just use an API?

Discussion

Have you used/developed MCP? What are some challenges you have encountered?

Exercise

Create an MCP server for accessing Wikipedia

Fine-tuning models for effective tool use

- Use/add special tokens for fine-grained control
- Ensure tool descriptions are clear and succinct, and the API is descriptively named.

Caution

For a given step in the workflow, models should be able to choose the correct tool, but this is very hard to achieve. Tool selection is a bottleneck.

Data Stores

- Internal - prompt repository, session memory, tools data
- External - databases, knowledge graphs, text files etc

More on this in Session 3!

Agent Loop Prompt

You are an AI agent with access to tools: [TOOL_DESCRIPTION]. Answer directly when no tool is needed. Otherwise, call tools using: <ToolCall> Tool: [TOOL_NAME] Input: [TOOL_ARGUMENTS] </ToolCall>

Tool results are returned as: <Observation>Tool: [TOOL_NAME] Result: [TOOL_RESULT] </Observation>

Use each Observation to decide whether to call another tool, ask a clarification, or finish. Do not invent tool results, call unnecessary tools, or combine tool calls with final answers. Ask before irreversible actions; if tools fail, retry or explain the limitation.

When finished, output only:<Answer>[FINAL_ANSWER] </Answer>

Guardrails

- PII (Personally Identifiable Information)
- Prompt Injection
- NSFW (Not Safe For Work) text
- Tone Check
- Web sanitization

Guardrails can be applied at both the input and output

Guardrails

- Bigger models have implicit guardrails that may already satisfy user requirements
- Addition of external guardrails are a drag on latency so make sure

Verifiers

- Verifiers can be added at required steps in the workflow
- Verifiers can check for solution quality, hallucinations, and more.
- Examples: schema validator, citation verifier, model-as-a-judge

Module 3

Memory and Retrieval

Types of information to retrieve

- External knowledge - structured and unstructured data
- Memory - conversational history, scratchpad, workflow logs
- Training examples - prompt repository, few-shot examples
- Tool data - tool list, tool documentation

Production-grade RAG

- Rewrite
- Retrieve
- Rerank
- Refine
- Insert
- Generate

RAG Pipeline

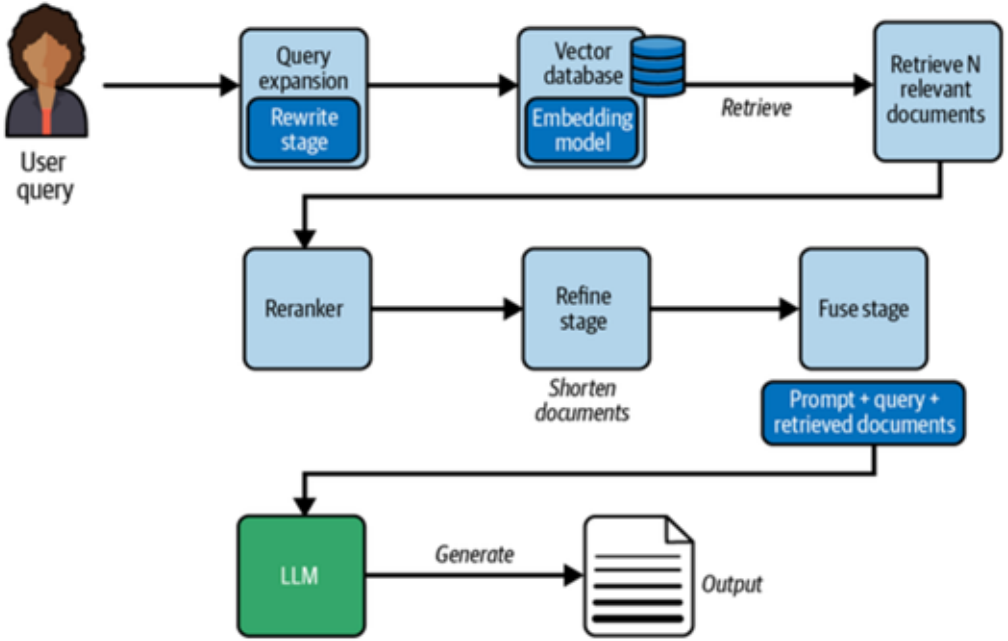


Figure 12-1. RAG pipeline

Rewrite

- Queries can be generated by humans or LLMs
- Vocabulary mismatch exists between query space and document space
- Typical approaches - pseudo-relevance feedback, query2doc, query2document2keyword

The uncanny effectiveness of doc2query

Sentence: ‘The company designs, manufactures, and markets smart phones, personal computers, and tablets’

doc2Query output: ‘What are the main activities and products of the company?

User Query: ‘What are the company’s products?’

Contextual Retrieval

- Prefix document chunks with informative context.
- The context is typically generated by an LLM

Retrieve

- Tried and tested technique - BM25 + Embedding-based

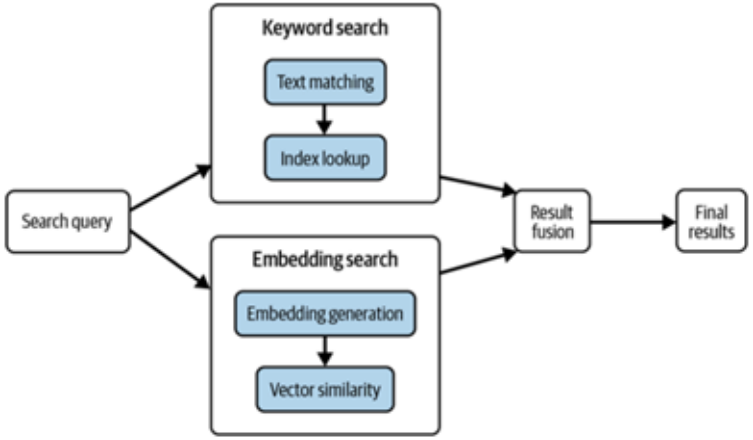


Figure 12-2. Hybrid search

Fine-tuning Embeddings

Similarity with respect to what?

Example: Negation

Sentence 1: ‘After analyzing the report, the company determined there is no imminent bankruptcy risk’

Sentence 2: ‘The company is struggling to stay solvent and bankruptcy is not out of the question’

Query: ‘Does the company have a bankruptcy risk?’

Generative Retrieval

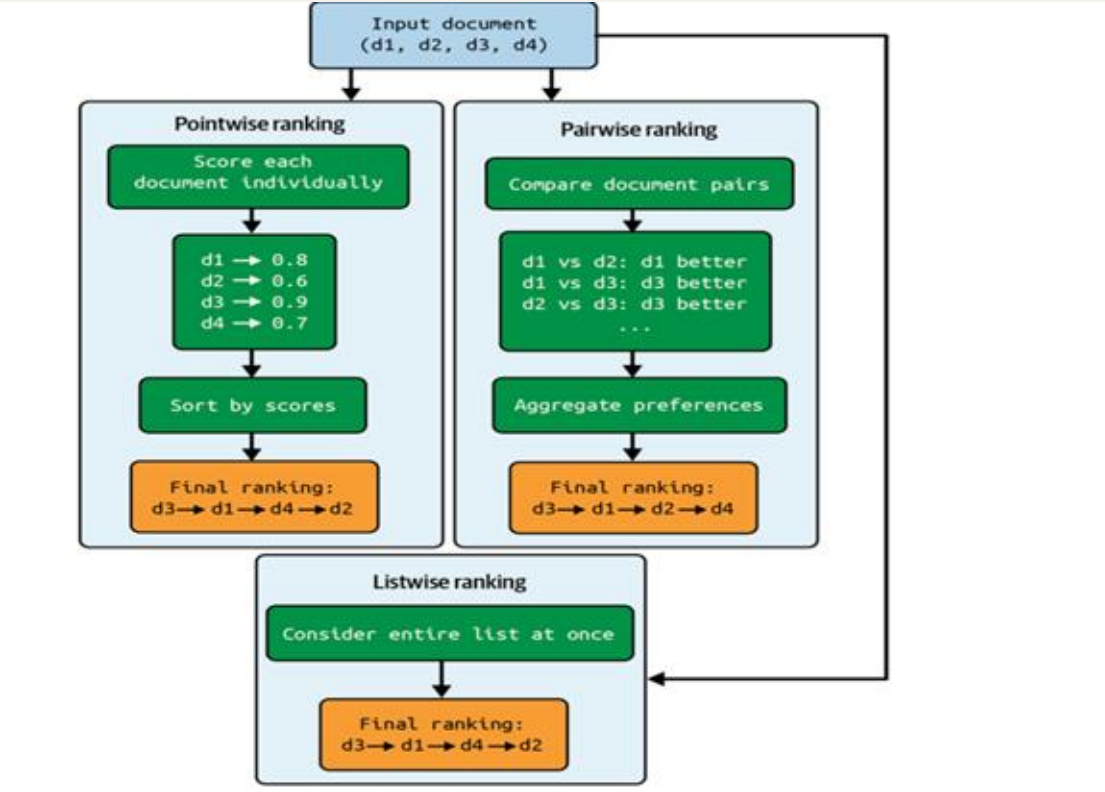
- LLM generates docid of a document in response to a query, the text corresponding to the docid is then retrieved
- Fine-tune model so that it retrieves the correct docid of document
- Constrained beam search so that only valid docids are generated during retrieval
- Docids can be salient keywords, metadata, doc identifiers, subset/prefix tokens

Rerank

- Cross-Encoders
- ColBERT
- Query Likelihood Model

QLM Prompt: “Generate a question that is most relevant to the given document <document content>”.

Reranking modes



Refine

- Summarization
- Chain-of-Note

Insert

- Due to Lost-in-the-middle phenomenon, the most promising documents can be inserted at the top and bottom of the context

Therefore, order of insertion = 1 3 5 7 9 10 8 6 4 2

15 newlines is all you need

- Separating retrieved docs from each other in the prompt can be hard sometimes.
- For models using ROPE and other relative positional encoding schemes, adding a lot of newline tokens helps with the separation

Context

- System Prompt
- User Prompt
- Retrieved context
- Conversation history
- Scratchpad
- Stored rules and preferences
- Workflow log
- Tool list and description
- Prompt/Instruction library

Not all of this can fit in the context window without degradation

Discussion

What does context engineering mean to you?

Context Engineering

- Ensuring that at any given time in the workflow, the model has all the necessary information in the context that is needed to solve the task
- Involves swapping in and out data from the context.

Context Engineering

- Short-term memory buffer (most recent messages)
- Compressed long-term summary
- Structured durable memory
- External knowledge chunks

Budget Enforcement Policy

When token budget is exceeded, execute in order:

1. Summarize older recent turns and remove raw messages from the context
2. Reduce number of items retrieved
3. Compress the summary

Exercise 3

Build a context engineering solution that dynamically swaps in and swaps out necessary context fed to the LLM during the processing of a task.

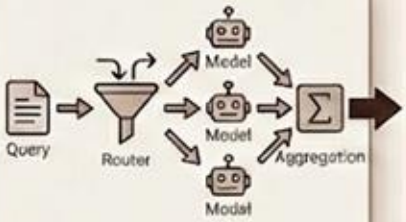
Module 4

Design Patterns

Multi-LLM Collaboration

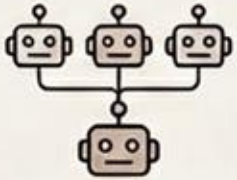
Text-based communication

Exchange of natural language messages between models



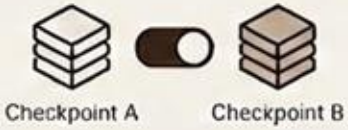
Routing & Aggregation

A router choosing between one or more models and aggregating the output in some form



Ensembling

Combining outputs from multiple models to improve performance



Checkpoint Switching

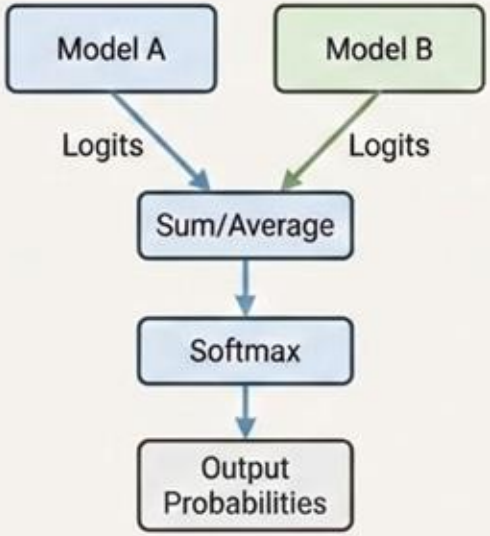
Dynamically selecting and loading different model checkpoints based on task or context

Multi-LLM design patterns

- Orchestrator-Worker
- Evaluator-Optimizer
- Planner-Executor

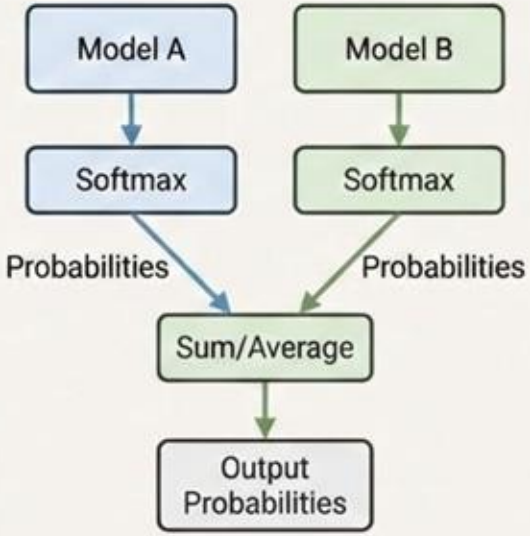
Ensembling (Late Fusion)

1. Logits Combination (Pre-Softmax)



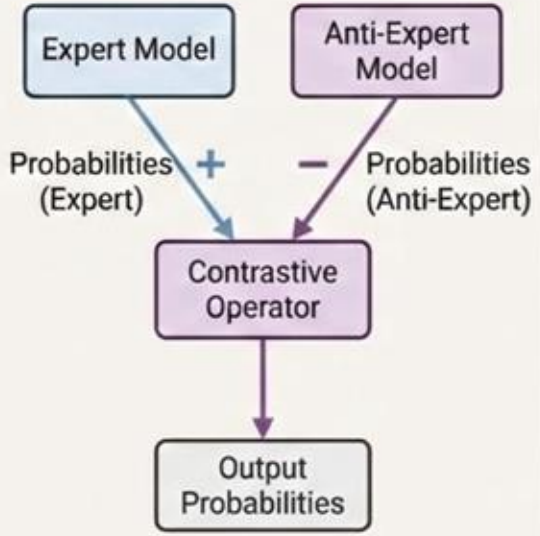
Combine logits from different models before computing softmax.

2. Probability Combination (Post-Softmax)



Combine softmax probabilities.

3. Contrastive Decoding (Anti-Experts)

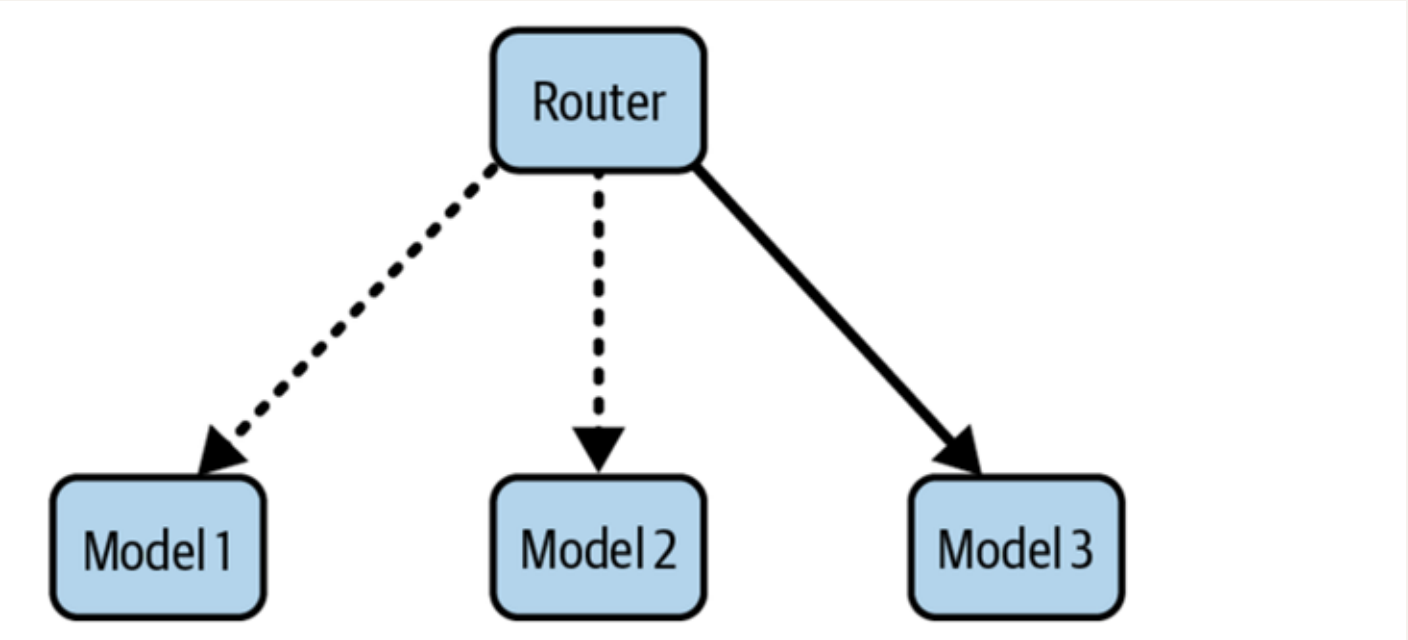


Contrastive decoding (adding anti-experts).

Other ensembling techniques

- Best-of-N
- Verifier-based selection
- Voting

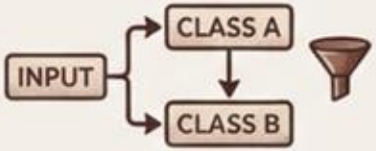
Routers



Routing Techniques

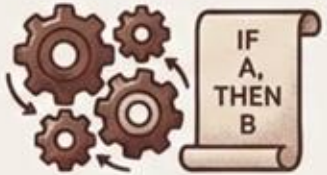
- Embedding similarity

- Classifier



- LLM-based

- Heuristics



Routing Dataset

Types of datasets for training a router:

1. Expert models' training datasets
2. Target task datasets
3. Generalist dataset

Cascades

1. Each input is fed to the small LLM.
2. If the small LLM makes an output prediction with a confidence level greater than a threshold, then we accept the output as the final output.
3. If the small LLM makes an output prediction with a confidence level that doesn't surpass the threshold, then we pass the input to the medium-sized model.
4. Similarly, if the medium-sized model makes an output prediction with a confidence level greater than a threshold, then we stop and accept this output as the final output.
5. However, if the medium LLM makes an output prediction with a confidence level that doesn't surpass the threshold, then we pass the input to the large model.
6. The large model generates the final output.

Agent Evaluation

- Final answer correctness
- Tool selection
- Tool arguments
- Policy adherence
- Appropriate clarification or escalation
- Number of steps/retries
- Latency and cost
- Recovery from tool failure

Need to invest in observability to facilitate evals and verification

Agent Observability

- Run and user identifiers.
- Model and configuration versions.
- Prompt, workflow and skill versions.
- Tool name, arguments and result status.
- State transitions.
- Routing and handoffs
- Guardrail decisions.
- Approval decisions
- Latency, tokens and cost.
- Citations and source tracking.
- Errors and retries.
- Final outcome and external state

Agent Security

- Prompt Injection
- Data Exfiltration
- Context poisoning
- Tool poisoning
- Malicious MCP servers

Building Production Agents

- Explicit workflow state
- Task and subtask status
- Checkpoints and resumability
- Queues and asynchronous work
- Timeouts and cancellation
- Retries with backoff
- Idempotency
- Compensating actions
- Concurrency and conflicting writes

Contact

- piesauce.substack.com
- https://x.com/piesauce
- https://www.linkedin.com/in/piesauce/